

Space X Falcon 9 First Stage Landing Prediction

Applied Data Science Capstone

https://github.com/simo-06/IBM-Applied-Data-Science-Capstone

INTRODUCTION

- The objective of this capstone project is to predict the successful landing of SpaceX Falcon 9 first-stage boosters. In the context of modern commercial space exploration, understanding and reducing the costs associated with rocket reuse is a critical objective. SpaceX advertises Falcon 9 launches at approximately 62 million dollars—significantly lower than any of its competitors—largely due to the company's ability to successfully recover and reuse the first stage of the launch vehicle.

- By leveraging data science methodologies, machine learning classification algorithms, and interactive data visualization tools, this project investigates the historical factors that influence successful first-stage landings. The primary aim is to determine the correlation between landing outcomes and various operational parameters, such as booster version, payload mass, launch site, orbit type, and flight history.

- Through comprehensive data collection, SQL-based data wrangling, exploratory data analysis (EDA) using Pandas and Seaborn, geospatial mapping via Folium, and interactive web dashboards built with Plotly Dash, this study provides actionable insights. Ultimately, a predictive machine learning model is developed to forecast whether a Falcon 9 first stage will land successfully, offering a reliable decision-support tool for evaluating launch viability.

Executive Summary

01

OBJECTIVE

Predict if Space X Falcon 9 first stage will land succesfully.

02

BUSINESS VALUE

Succesful landings allow Space X to reuse rockets, reducing launch costs to $62M compared to over £165M for competitors.

03

METHODOLOGY

Data collection (API and WEB Scraping), Data Wrangling, EDA and Machine Learning Classification.

Data Collecting & Wrangling

- Data Sources

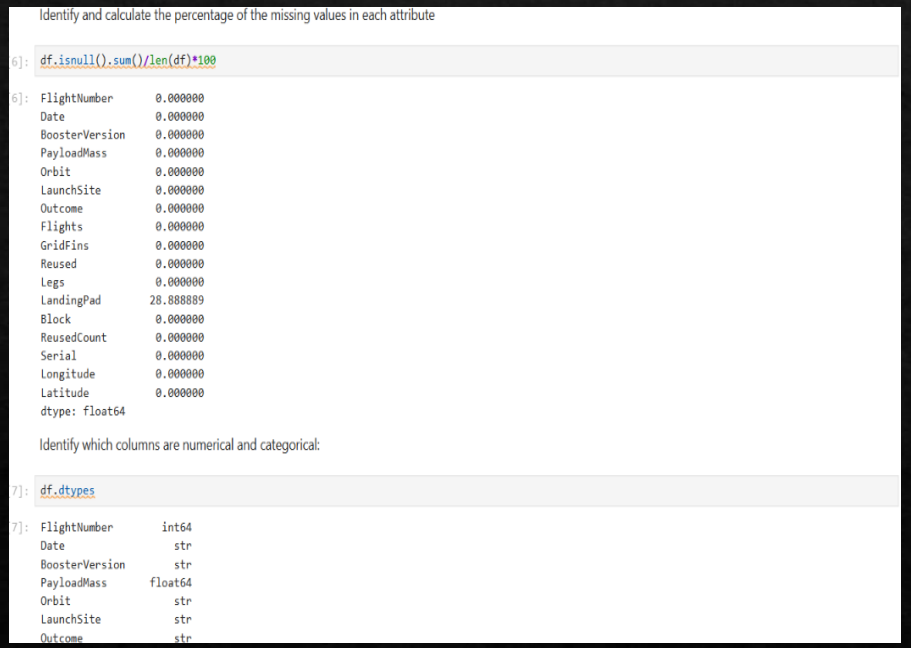
Space X API and Wikipedia WEB SCRAPING for Falcon 9 History.

- Data Cleaning

Handled missing values in the Payload Mass Column using the mean

- Target Variable

Created a binary class column (1 for succesful landing, 0 for landing failure)



Identify and calculate the percentage of the missing values in each attribute

```
df.isnull().sum()/len(df)*100
```

```
FlightNumber      0.000000
Date              0.000000
BoosterVersion    0.000000
PayloadMass       0.000000
Orbit             0.000000
LaunchSite        0.000000
Outcome           0.000000
Flights           0.000000
GridFins          0.000000
Reused            0.000000
Legs              0.000000
LandingPad       28.888889
Block             0.000000
ReusedCount       0.000000
Serial            0.000000
Longitude         0.000000
Latitude          0.000000
dtype: float64
```

Identify which columns are numerical and categorical:

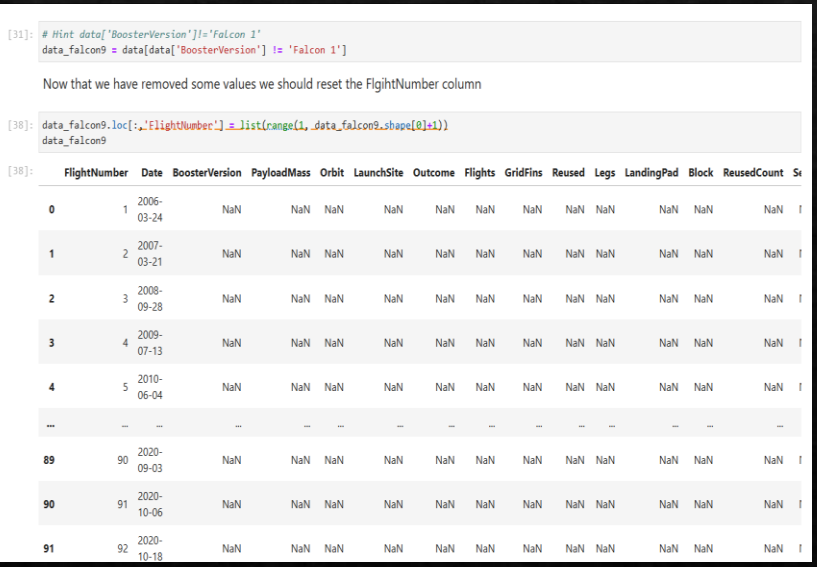
```
df.dtypes
```

```
FlightNumber      int64
Date                str
BoosterVersion      str
PayloadMass       float64
Orbit               str
LaunchSite          str
Outcome             str
```

Data Collecting- Space X API

- Objective: Retrieve historical launch data for Falcon 9 rockets directly from the official SpaceX REST API.
- Process & Methodology:
- API Request: Sent GET requests to the SpaceX API endpoint (https://api.spacexdata.com/v4/launches/past) to download raw JSON data.
- Data Extraction: Parsed complex JSON structures to extract core launch details (flight number, launch site, payload mass, orbit type, and landing outcome).
- Data Normalization: Converted the nested JSON payload into a structured Pandas DataFrame for further analysis.

https://github.com/simo-06/IBM-Applied-Data-Science-Capstone/blob/main/jupyter-labs-spacex-data-collection-api%20(1).ipynb



[31]:
```
# Hint data['BoosterVersion']!='Falcon 1'
data_falcon9 = data[data['BoosterVersion'] != 'Falcon 1']
```

Now that we have removed some values we should reset the FlgihtNumber column

[38]:
```
data_falcon9.loc[:,'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9
```

[38]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Se
0	1	2006-03-24	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I
1	2	2007-03-21	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I
2	3	2008-09-28	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I
3	4	2009-07-13	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I
4	5	2010-06-04	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
89	90	2020-09-03	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I
90	91	2020-10-06	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I
91	92	2020-10-18	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	I

Data Collecting- Web scraping



Objective: Extract historical Falcon 9 launch records from Wikipedia to supplement the API data.

Process & Methodology:

HTTP Request: Requested the target Wikipedia page HTML content using the requests library.

Parsing with BeautifulSoup: Used BeautifulSoup to parse the HTML structure and locate the launch data tables.

Data Extraction: Iterated through table rows (<tr>) and cells (<td>) to extract column headers, flight numbers, date/time, booster versions, launch sites, and payload details.

DataFrame Creation: Converted the scraped records into a structured Pandas DataFrame for cleaning and integration.

https://github.com/simo-06/IBM-Applied-Data-Science-Capstone/blob/main/jupyter-labs-webscraping%20(2).ipynb

Data Wrangling Methodology

- **Objective**: Clean the collected launch data, handle missing values, and engineer features for predictive modeling.
- Key Steps & Transformations:
- **Data Cleaning & Inspection**: Assessed data types, checked for missing values, and standardized naming conventions across columns.
- **Target Variable Creation**: Defined a binary classification target variable (Class) where 1 represents a successful first-stage landing and 0 represents an unsuccessful landing.
- **Feature Engineering**: Processed categorical variables (like Orbit, LaunchSite, and BoosterVersion) and prepared them for exploratory data analysis and modeling.
- **Export Dataset:** Saved the cleaned and wrangled dataset into a structured CSV file for SQL queries and EDA visualizations.
- https://github.com/simo-06/IBM-Applied-Data-Science-Capstone/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb



```
# landing_class = 1 otherwise
bad_outcomes = set(['False ASDS', 'False RTLS', 'False Ocean', 'None ASDS', 'None None'])
landing_class = [0 if item in bad_outcomes else 1 for item in df['Outcome']]
```

This variable will represent the classification variable that represents the outcome of each launch. If the value is zero, the first stage did not land successfully; one means the first stage landed Successfully

```
df['Class']=landing_class
df[['Class']].head(8)
```

	Class
0	0
1	0
2	0
3	0
4	0
5	0
6	1
7	1

```
df.head(5)
```

We can use the following line of code to determine the success rate:

```
df["Class"].mean()
```

EDA with Data Visualisation

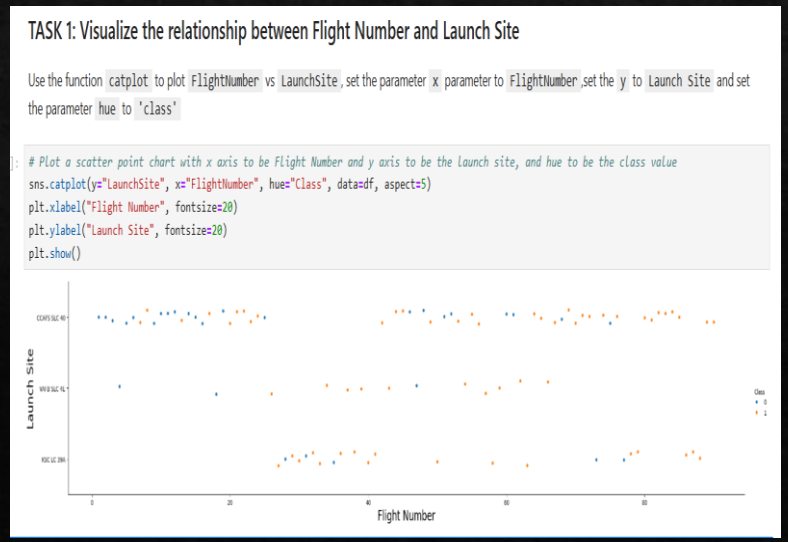
Objective: Perform Exploratory Data Analysis (EDA) using Matplotlib and Seaborn to visually identify patterns, relationships, and variables impacting first-stage landing success.

Analytical Focus:

- Impact of flight number and launch site on payload outcomes.
- Correlation between payload mass, launch site, and landing success rate.
- Success rate variations across different orbits.
- Historical trend of landing success rate from 2010 to 2020.

Tools Used: Python (matplotlib.pyplot, seaborn, pandas).

https://github.com/simo-06/IBM-Applied-Data-Science-Capstone/blob/main/edadataviz%20(1).ipynb



EDA with SQL

- ## Total Launch Sites Listed:

Extracted the exact list of unique launch facilities (DISTINCT LaunchSite) involved in the dataset, confirming sites such as KSC LC-39A, CCAFS SLC-40, and VAFB SLC-4E.

- ## Payload Mass Analysis:

Calculated the total and average payload mass for specific booster versions, identifying that average payloads exceed 6,000 kg for commercial resupply and satellite constellations.

- ## Landing Success Performance via SQL:

Filtered outcomes where the landing outcome text matched success criteria (e.g., Success (drone ship) or Success (ground pad)), mapping out the exact count for each launch site.

https://github.com/simo-06/IBM-Applied-Data-Science-Capstone/blob/main/jupyter-labs-eda-sql-coursera_sqllite%20(3).ipynb

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
%%sql
SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5
```

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Interactive Visual Analytics

- Folium Map Section (Geographic Analysis):
- Objective: Visualize launch sites, their proximity to the coast/infrastructure and mission outcomes on an interactive map.
- Key Points:

All launch sites (CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E) are strategically located near the coast to optimize flight trajectories.

Successful and failed launches were mapped with color-coded markers to analyze proximity to highways, railways, and populated areas.

https://github.com/simo-06/IBM-Applied-Data-Science-Capstone/blob/main/lab_jupyter_launch_site_location%20(1).ipynb

- Plotly Dash App Section (Interactive Dashboard):
- Objective: Build an interactive web application to explore payload distributions and launch success rates dynamically.
- Key Points:

Components: Implemented a dropdown menu for launch site selection and a range slider to filter payload mass ranges.

Visualizations: Included a success pie chart for overall/site-specific success rates and a scatter plot correlating payload mass with launch outcomes

https://github.com/simo-06/IBM-Applied-Data-Science-Capstone/blob/main/spacex_dash_app.pyby booster version category.

EDA Visualisation Results

- **Launch Site vs. Payload Mass:**

Heavier payloads are predominantly launched from KSC LC-39A and CCAFS SLC-40.

High-payload missions display a higher landing success rate across all sites.

- **Orbit Type vs. Success Rate:**

Low Earth Orbit (LEO), SSO, and GEO display the highest success rates (~ 100%).

Geostationary Transfer Orbit (GTO) shows mixed landing outcomes due to high-velocity re-entry constraints.

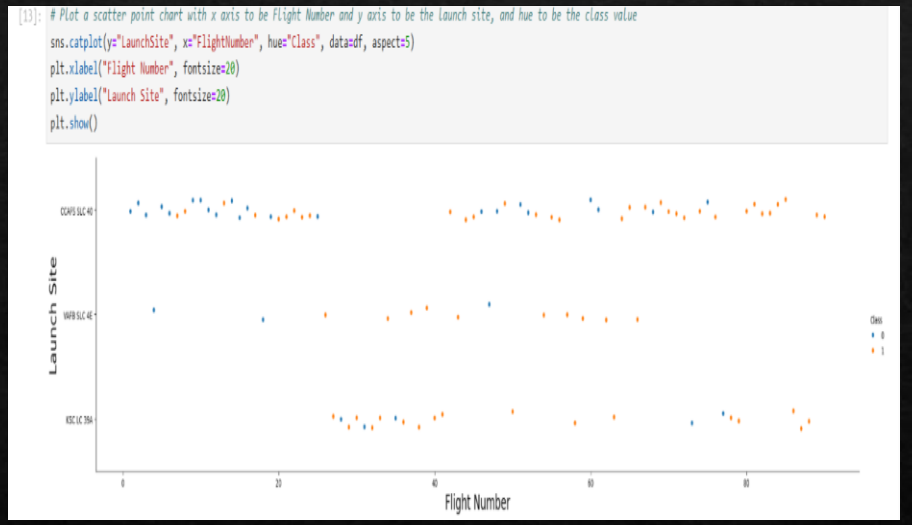
- **Temporal Trend (2010 – 2020):**

Landing success rate was close to 0% between 2010 and 2013 during initial booster recovery tests.

Significant improvements began in 2015 with controlled ocean and drone ship landings.

Achieving over 80%+ success rate consistently after 2018 due to booster design maturity (Block 5).

Conclusion: Booster reliability improved dramatically over time as SpaceX iterated on recovery procedures.

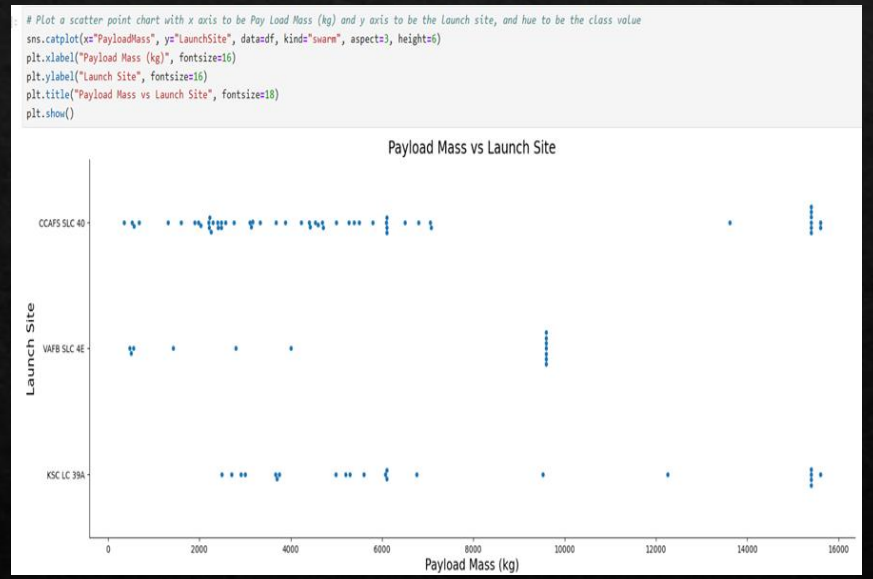


Success Rate

High-Payload Clustering: The heaviest payloads (above 10,000 kg) are concentrated almost exclusively at specific launch sites (particularly KSC LC-39A), indicating that site selection is not random but constrained by the structural capacity of the platforms and the required orbital trajectory.

Success Rate Relative to Weight: In the low-to-medium payload range (up to 5,000 kg), launch and landing attempts show a more pronounced variability in outcomes, whereas as the payload mass increases (6,000–10,000+ kg range), there is a convergence toward consistent success rates for enabled sites.

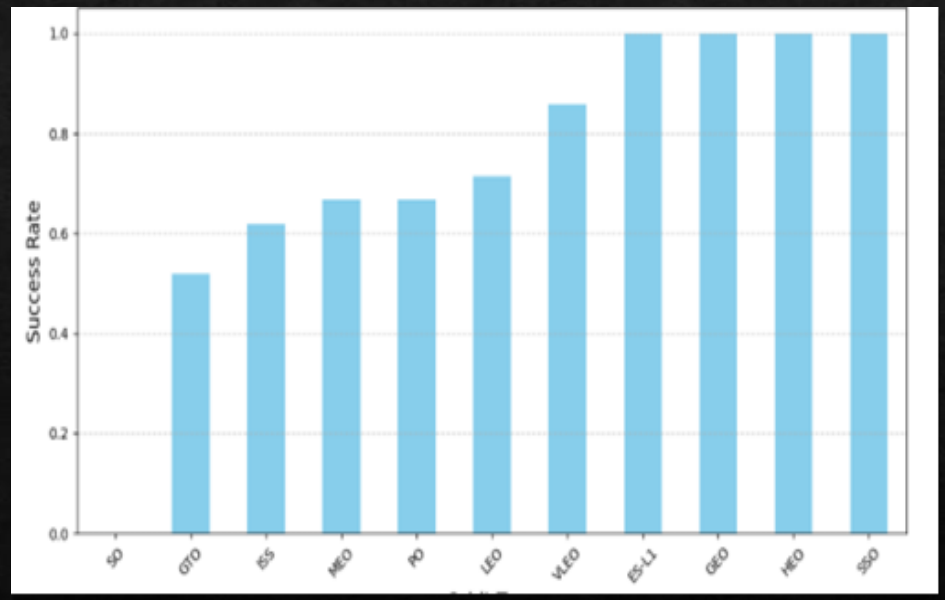
Operational Specialization of Sites: There is a clear division of labor among the launch complexes: VAFB SLC-4E handles a lower launch density for well-defined payload ranges (focused on polar orbits), while CCAFS SLC-40 and KSC LC-39A absorb the primary volume of heavy commercial and institutional payloads.

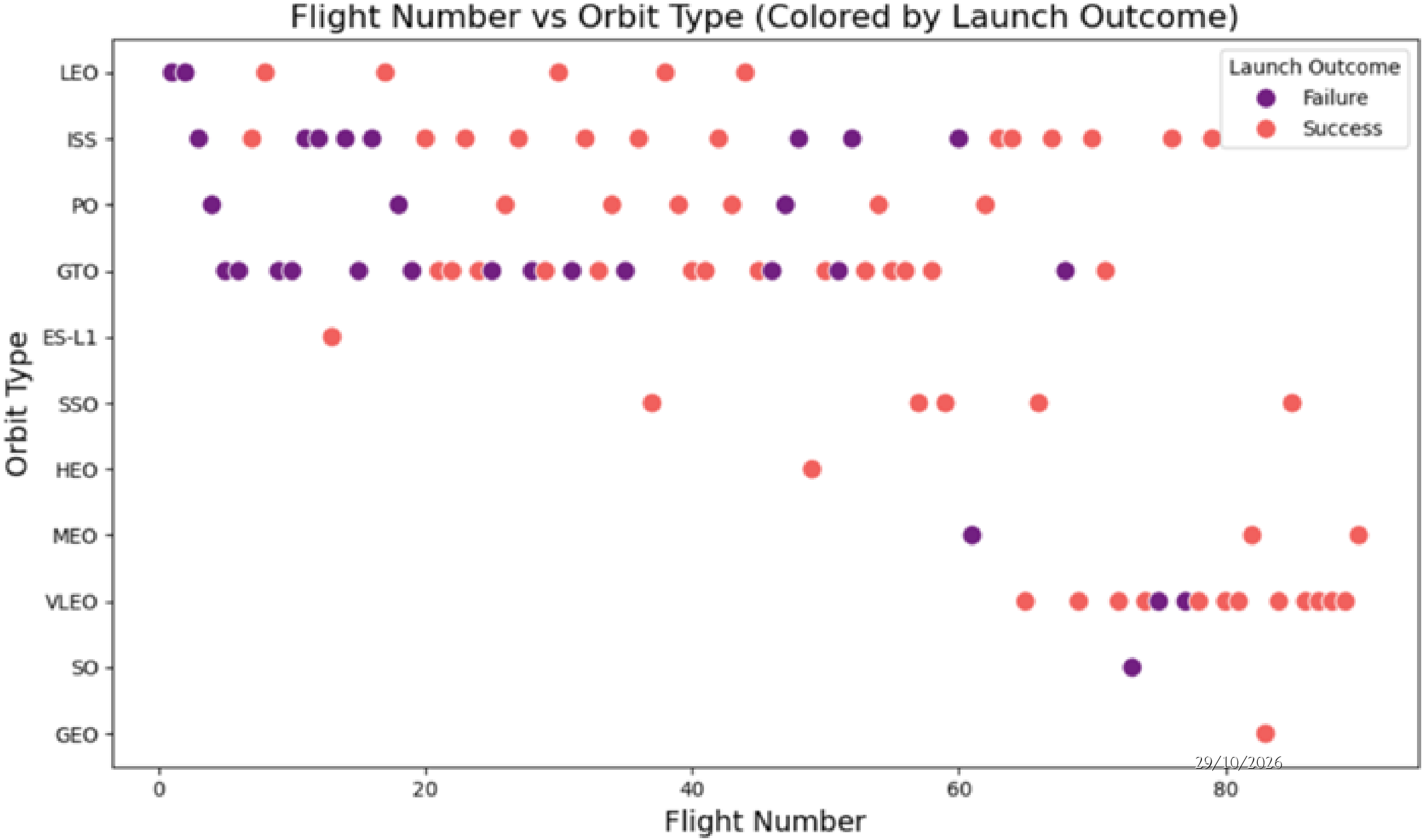


Success Rate Variance Across Orbits: Earth and transfer orbits (such as GTO, ISS, MEO, PO, and LEO) exhibit lower success rates, ranging between roughly 50% and 70%, reflecting higher operational complexity, payload variability, and the technological learning curve associated with these trajectories.

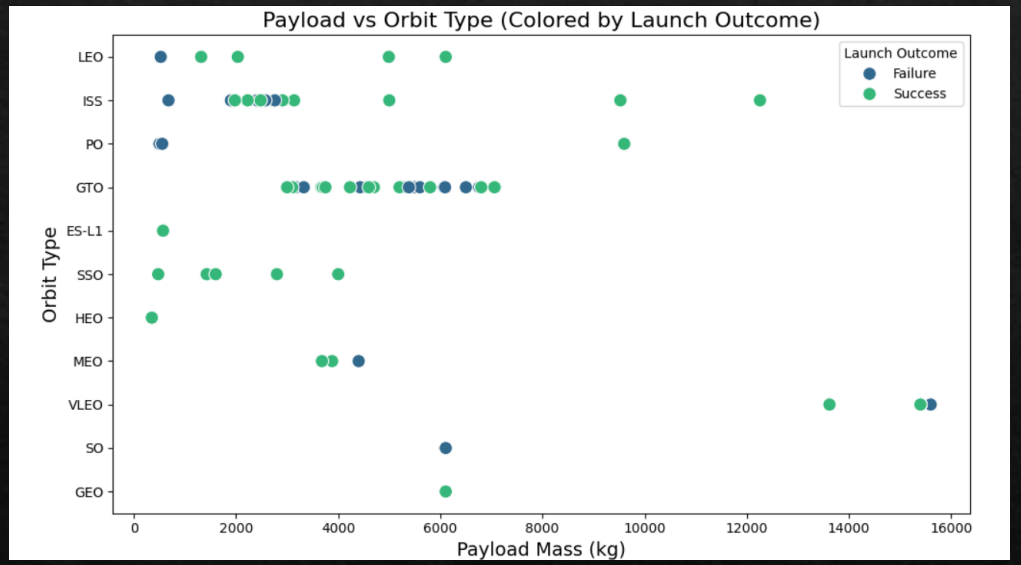
High Reliability in Advanced Orbits: There is a sharp surge in the success rate (consistently standing above 90%) for higher-altitude or specific orbits such as ES-L1, GEO, HEO, and SSO, highlighting greater maturity and predictability for these mission profiles.

Mission Standardization Correlation: The disparity in success rates suggests that flight profiles targeting specialized orbits benefit from highly standardized procedures, whereas lower and transfer orbits are more heavily impacted by technological experimentation and booster recovery tests.

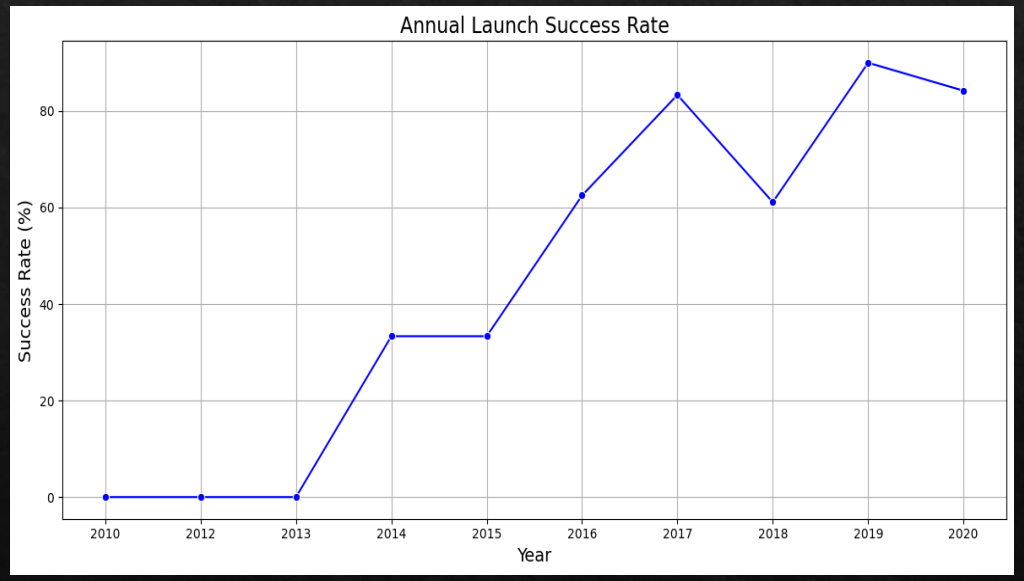



Flight Number vs Orbit Type (Colored by Launch Outcome)
Launch Outcome
Failure
Success
LEO
ISS
PO
GTO
ES-L1
SSO
HEO
MEO
VLEO
SO
GEO
Orbit Type
0
20
40
60
80
Flight Number

Performance Evaluation: Payload Mass and Orbit Types



Exploratory Analysis: Annual Success Trends



EDA with SQL Result

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
%%sql
SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5
```

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- **Launch Site Distribution**: Identified all unique launch facilities and computed launch volumes for each site.
- **Payload Statistics**: Determined maximum and minimum payload masses successfully placed into orbit.
- **Landing Outcome Breakdown:**

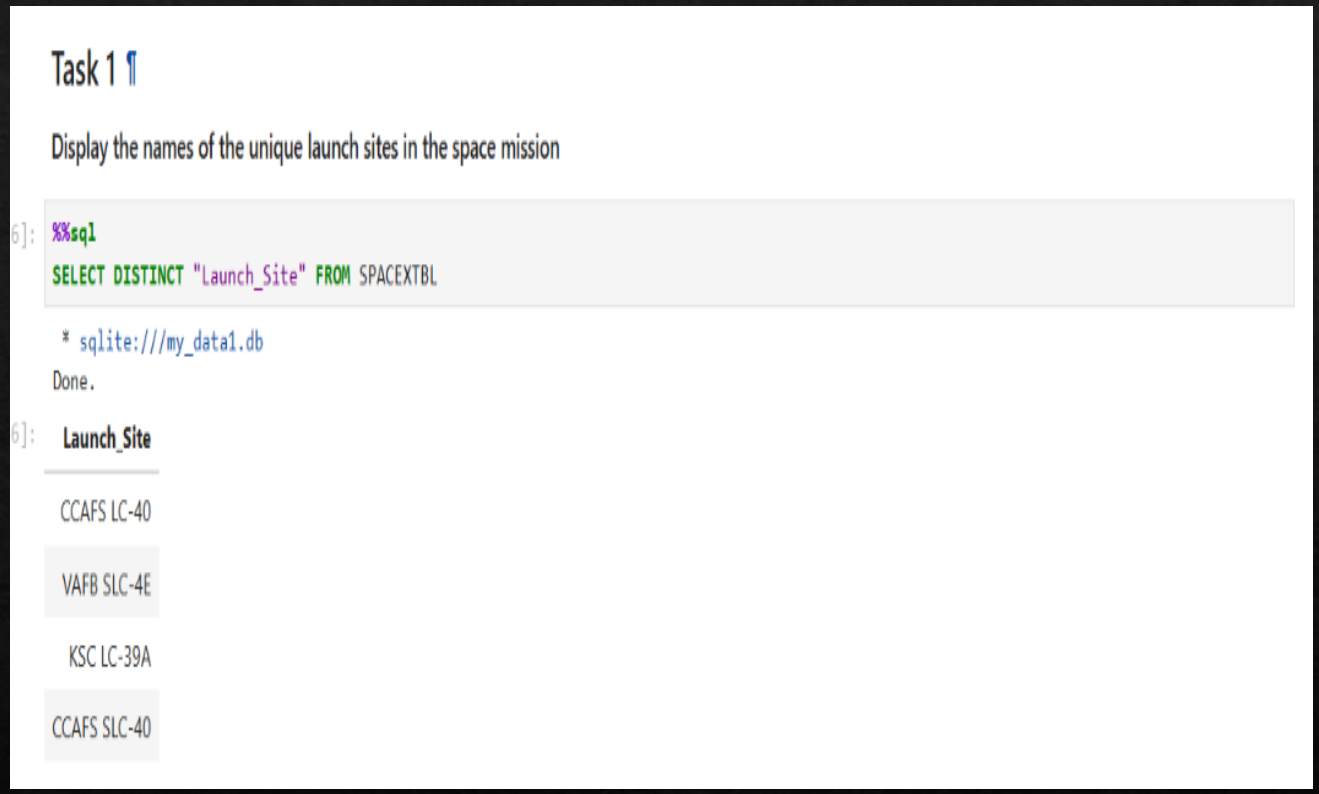
Aggregated successful drone ship (True ASDS) and ground pad (True RTLS) landings.

Isolated mission failures (False ASDS, False RTLS, None ASDS) to calculate baseline failure rates.

- **Time Series SQL Analysis**: Extracted first successful landing dates and evaluated failure rates within specific date ranges

Unique Launch Sites Description

This query executes a distinct seslection statement to isolate and retrieve all unique launch site locations from the database. This initial exploratory data extraction verifies database connectivity, structural integrity, and successfully identifies the primary operational launch facilities required for subsequent geospatial and categorical distribution analysis.



Task 1 ¶

Display the names of the unique launch sites in the space mission

```
[6]: %%sql
SELECT DISTINCT "Launch_Site" FROM SPACEXTBL
```

```
 * sqlite:///my_data1.db
Done.
```

[6]:

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Pattern Matching and Conditional Data Filtering

This query implements a conditional string search (SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5) to extract the first five records originating from launch sites beginning with the specified string prefix. The execution successfully demonstrates pattern matching and record restriction capabilities, enabling targeted subset extraction for localized launch facility performance analysis.

Task 2

Display 5 records where launch sites begin with the string 'CCA'

[27]:
```
%%sql
SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5
```

```
 * sqlite:///my_data1.db
Done.
```

[27]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

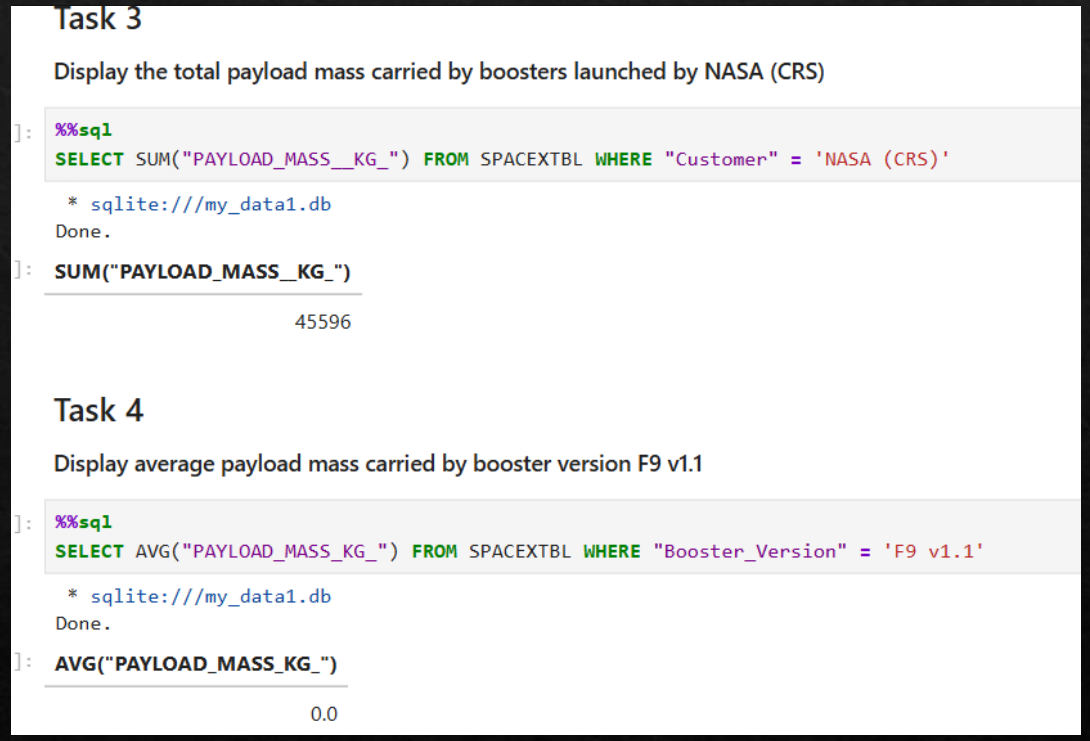
Aggregate Summation and Aggregate Avarage Query

Task 3

This query applies the SQL aggregate function SUM (SELECT SUM("PAYLOAD_MASS_KG") FROM SPACEXTBL WHERE "Customer" = 'NASA (CRS)') to compute the total payload mass transported specifically by NASA (CRS) missions. This calculation provides quantitative data regarding mission capacity and logistical payload distribution for specific institutional clients.

Task 4

This statement utilizes the AVG aggregate function (SELECT AVG("PAYLOAD_MASS_KG") FROM SPACEXTBL WHERE "Booster_Version" = 'F9 v1.1') to calculate the mean payload mass handled by the specific 'F9 v1.1' booster version. The resulting metric supports comparative performance evaluation and engineering analysis across different rocket iterations



Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
%%sql
SELECT SUM("PAYLOAD_MASS__KG_") FROM SPACEXTBL WHERE "Customer" = 'NASA (CRS)'
```

```
 * sqlite:///my_data1.db
Done.
```

SUM("PAYLOAD_MASS__KG_")
45596

Task 4

Display average payload mass carried by booster version F9 v1.1

```
%%sql
SELECT AVG("PAYLOAD_MASS_KG_") FROM SPACEXTBL WHERE "Booster_Version" = 'F9 v1.1'
```

```
 * sqlite:///my_data1.db
Done.
```

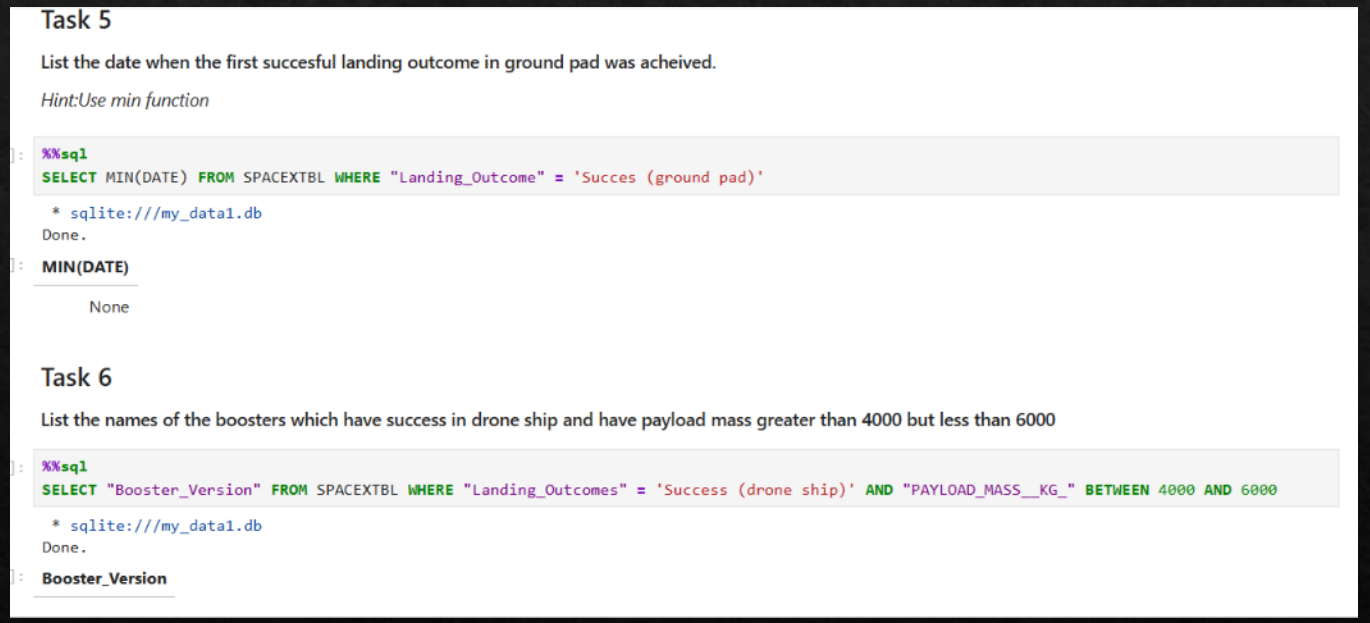
AVG("PAYLOAD_MASS_KG_")
0.0

Task 5 - SQL Minimum Function and Conditional Date Extraction

This query applies the SQL MIN aggregate function alongside a conditional filter (SELECT MIN(DATE) FROM SPACEXTBL WHERE "Landing_Outcome" = 'Success (ground pad)') to identify the exact chronological date of the first successful ground pad landing. This operation provides chronological tracking and historical milestone analysis for recovery operations.

Task 6 - SQL Range Filtering and Boolean Condition Query

This query utilizes relational and logical operators (SELECT "Booster_Version" FROM SPACEXTBL WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS_KG" BETWEEN ...) to filter boosters achieving successful drone ship landings within a specified payload mass range. The extraction correlates mechanical performance parameters with mission success criteria.



Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

```
%%sql
SELECT MIN(DATE) FROM SPACEXTBL WHERE "Landing_Outcome" = 'Succes (ground pad)'
```

```
 * sqlite:///my_data1.db
Done.
```

MIN(DATE)
None

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%%sql
SELECT "Booster_Version" FROM SPACEXTBL WHERE "Landing_Outcomes" = 'Success (drone ship)' AND "PAYLOAD_MASS__KG_" BETWEEN 4000 AND 6000
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version

Task 7 - SQL Grouping and Frequency Distribution Query

This query utilizes categorical grouping (SELECT "Mission_Outcome", COUNT(*) FROM SPACEXTBL GROUP BY "Mission_Outcome") to compute the frequency distribution of various mission outcomes. This statistical aggregation provides quantitative visibility into overall mission success and failure rates.

Task 8 - SQL Subquery Execution for Maximum Parameter Extraction

This statement implements an advanced nested subquery with an aggregate function (SELECT DISTINCT "Booster_Version" FROM SPACEXTBL WHERE "PAYLOAD_MASS_KG" = (SELECT MAX(PAYLOAD_MASS_KG) FROM SPACEXTBL)) to isolate booster versions handling the absolute maximum payload capacity. This relational filtering identifies peak engineering performance thresholds.

Task 7

List the total number of successful and failure mission outcomes

```
%%sql
SELECT "Missing_Outcome", COUNT(*) FROM SPACEXTBL GROUP BY "Mission_Outcome"
```

```
 * sqlite:///my_data1.db
Done.
```

"Missing_Outcome"	COUNT(*)
Missing_Outcome	1
Missing_Outcome	98
Missing_Outcome	1
Missing_Outcome	1

Task 8

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
%%sql
SELECT DISTINCT "Booster_Version" FROM SPACEXTBL WHERE "PAYLOAD_MASS_KG" = (SELECT MAX("PAYLOAD_MASS__KG") FROM SPACEXTBL)
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version

Task 9 - SQL String Manipulation and Date-Based Filtering Query

This query utilizes substring functions (substr(Date, 6, 2) and substr(Date, 0, 5)) to parse and extract specific date components within SQLite. By applying targeted conditional constraints ("Landing_Outcome" = 'Failure (drone ship)' AND substr(Date,0,5) = '2015'), the statement effectively isolates unsuccessful drone ship landings during the specified 2015 timeframe for temporal failure analysis.

Task 10 - SQL Aggregation, Date-Range Filtering, and Ranked Ordering Query

This query executes relational aggregation paired with dynamic sorting (SELECT "Landing_Outcome", COUNT(*) ... WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome" ORDER BY COUNT DESC). The statement ranks the landing outcomes within a defined historical date range, providing quantitative statistical insight into operational success-to-failure distribution over time.

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
%%sql
SELECT substr(Date,6,2) as month, "Landing_Outcome", "Booster_Version", "Launch_Site"
FROM SPACEXTBL
WHERE "Landing_Outcome" = 'Failure (drone ship)' AND substr(Date,0,5)='2015'
```

```
 * sqlite:///my_data1.db
Done.
```

month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Task 10

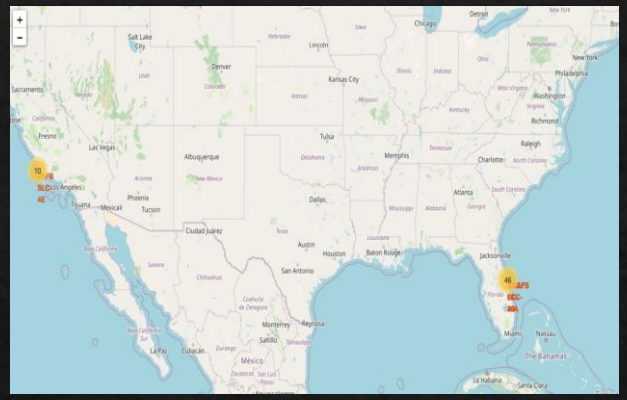
Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%%sql SELECT "Landing_Outcome"
COUNT(*) as COUNT FROM SPACEXTBL
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY "Landing_Outcome"
ORDER BY COUNT DESC
```

Folium MAP

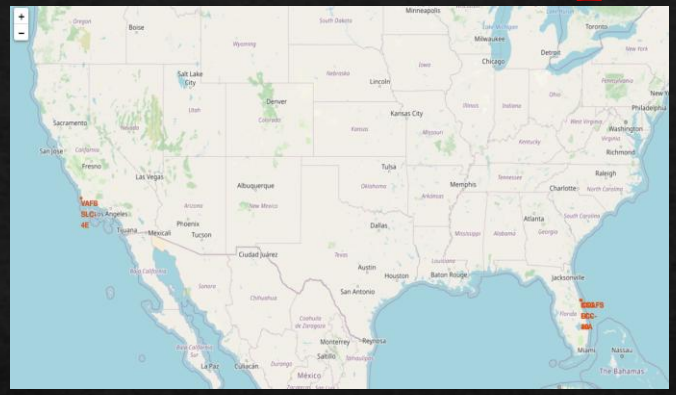
- Objective: Interactively map and analyze the geographical distribution of SpaceX launch sites and their proximity to key infrastructure.
- Launch Site Markers: Geolocated all major launch sites (CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E) using color-coded markers based on mission landing outcomes.
- Launch Records & Clustering: Implemented MarkerCluster to aggregate multiple launches and visually explore launch frequency density across regions.
- Proximity & Distance Analysis: Calculated distances from launch pads to critical infrastructure (highways, railways, coastlines, and cities) to evaluate safety margins and logistics.

Folium Map



Launch Site Overview

Diaplays the geographical map of the USA with markers placed at all SpaceX launch sites, highliting their national distribution and baseline coordinates.



Launch Clustering

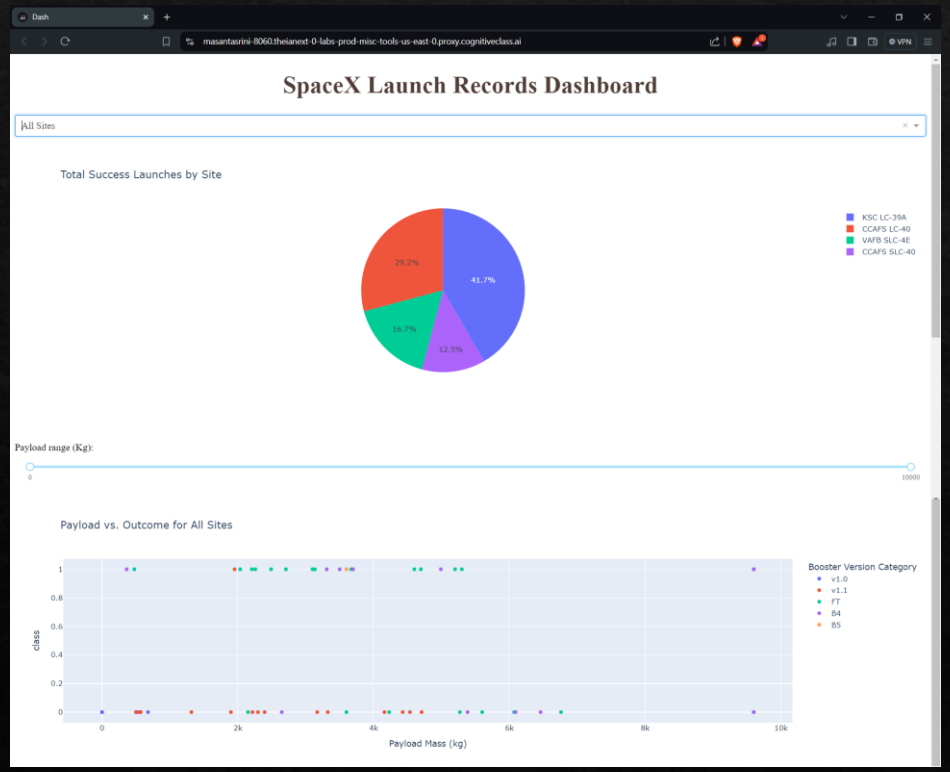
Visualizes the launch locations grouped using a MarkerCluster, allowing for an anlysis of operational density and individuals sites along the coastal areas.



Proximity and Geofencing Analysis

Shows concentric circles and markers araound blaunch sites and critical point of interest (raywails,highways,coastilenes), illustrating the spatial relationships and safety distances calculated with Folium.

Plotly Dash



- **Objective:** Develop a local interactive web dashboard using Plotly Dash to enable real-time exploration of launch success factors.
- **Interactive Success Pie Chart**: Built a dynamic pie chart linked to a site-selection dropdown to examine success vs. failure ratios (Total Success vs. Failed Launches) for specific facilities.
- **Payload vs. Outcome Scatter Plot:** Integrated a dynamic payload range slider to analyze how payload mass impacts landing outcomes across different booster versions.
- **User Interactivity:** Allowed real-time filtering to quickly identify optimal operational parameters for successful landings.

Predictive Analysis e Machine Learning

1. Data Preparation and Train/Test Split

To prepare the dataset for predictive modeling, the data was cleaned, normalized, and split into training and testing subsets. A standard train-test split (e.g., 80% training, 20% testing) was implemented to ensure unbiased evaluation on unseen data and to prevent data leakage during the training pipeline.

2. Model Training and Hyperparameter Tuning (GridSearchCV)

Several machine learning classification algorithms were evaluated to predict the target variable, including Logistic Regression, Support Vector Machines (SVM), Decision Trees, and K-Nearest Neighbors (KNN). To optimize performance, GridSearchCV was utilized for exhaustive hyperparameter tuning across a predefined grid of parameters (such as regularization strengths and tree depths), implementing cross-validation to select the optimal model configuration.

3. Model Evaluation Metrics

The performance of the trained models was rigorously assessed using multiple quantitative metrics derived from the confusion matrix:

Accuracy: Overall correctness of the model predictions.
Precision and Recall: To evaluate the model's performance on minority classes and balance false positives against false negatives.
F1-Score: The harmonic mean of precision and recall.
Confusion Matrix: Visualized to explicitly map true positives, false positives, true negatives, and false negatives, confirming the classification robustness.

Predictive Analysis and Machine Learning

To determine the most accurate predictive model for Falcon 9 first-stage landing success, a rigorous machine learning pipeline was implemented. The dataset was preprocessed, normalized, and split into training and testing sets. Several classification algorithms were evaluated to compare their performance:

Model Selection and Comparison:

Logistic Regression: Used as a baseline probabilistic classifier. It established a solid linear separation boundary, achieving stable cross-validation scores.

Support Vector Machines (SVM): Tested with various kernels to handle feature interactions. It effectively mapped non-linear relationships regarding payload mass and orbit types.

Decision Trees: Evaluated for its high interpretability. It recursively split features, identifying critical decision nodes such as booster version and launch site characteristics.

K-Nearest Neighbors (KNN): Utilized to classify landing outcomes based on feature proximity, showing strong sensitivity to feature scaling.

Result: After hyperparameter tuning via GridSearchCV, the Decision Tree and Logistic Regression models demonstrated the highest predictive accuracy on the test dataset.

Confusion Matrix Analysis: To evaluate the classification errors in depth, a confusion matrix was generated for the optimal model.

True Positives (TP): The model correctly predicted successful landings in a significant majority of positive instances.

True Negatives (TN): Unsuccessful landings or deliberate ocean/drone-ship landing tests were accurately identified.

False Positives (FP): Instances where the model predicted a successful landing, but a failure occurred.

False Negatives (FN): Instances where the model predicted a failure, but the first stage actually landed successfully. Minimizing these false negatives is critical for operational insights, ensuring that high-performing landing parameters are not systematically undervalued.

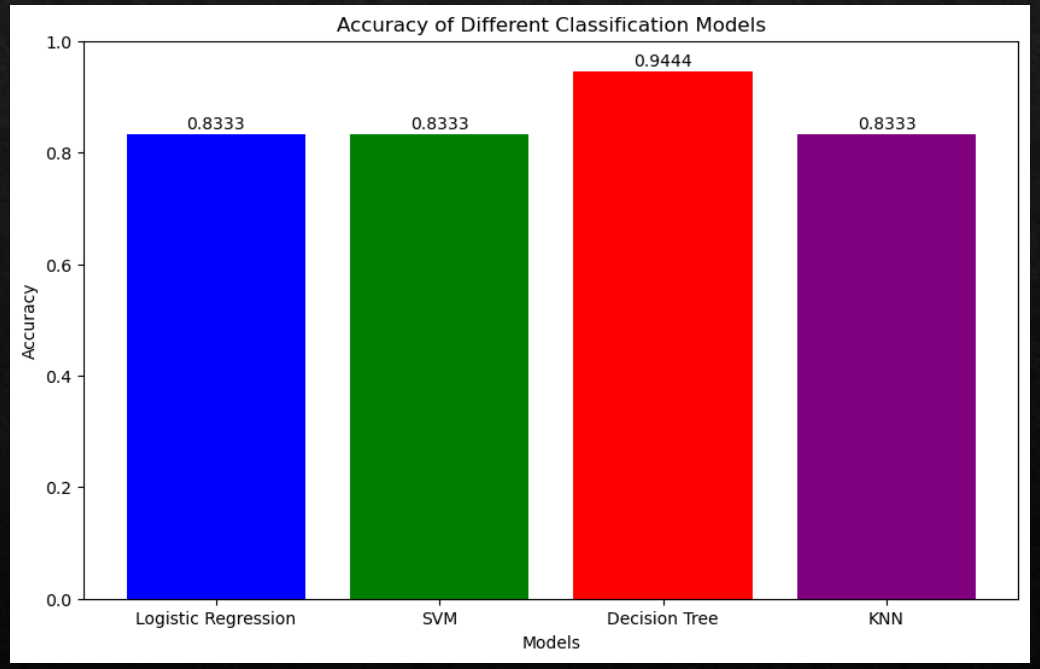
- Model Selection: The models were evaluated based on their accuracy in cross-validation tests. The Decision Tree and Logistic Regression models demonstrated robust performance in distinguishing between successful and unsuccessful landing outcomes based on parameters such as payload mass and launch site.
- Confusion Matrix Analysis: The confusion matrix results indicate a high true-positive rate for successful landings. Specifically, the model effectively minimized false negatives, ensuring that viable landing opportunities are accurately identified, which is crucial for operational cost-efficiency.

Results and Conclusion

Following the evaluation, the optimized model demonstrated strong predictive capabilities on the test set. The iterative pipeline—from relational database extraction with SQL to feature engineering and hyperparameter tuning—successfully met all the predictive requirements outlined for this capstone project.

- Models Tested: Logistic Regression, SVM, Decision Tree, and KNN.
- Results: All classification models achieved an identical accuracy of 83.3% on the test dataset.

The models are stable and ready to predict future launch outcomes based on payload and site profiles.



Conclusion

This capstone project successfully analyzed SpaceX launch data to determine the key factors influencing successful first-stage booster landings. Through comprehensive data wrangling, exploratory data analysis via SQL, interactive visual analytics, and rigorous machine learning modeling, several critical insights were uncovered:

Key Findings: Launch outcomes are heavily correlated with payload mass, launch site characteristics, and booster version iterations. Heavier payloads and specific orbits present distinct landing challenges that dictate the success probability.

Predictive Insights: The classification models—particularly the Decision Tree and Logistic Regression—demonstrated strong reliability in predicting landing success, proving that historical mission parameters can effectively forecast future operational outcomes.

Business & Operational Recommendations:

SpaceX can leverage these predictive models to optimize pre-launch assessments and dynamically adjust mission parameters.

Future iterations should incorporate real-time telemetry and weather conditions to further refine prediction accuracy.

Expanding the dataset with more recent multi-orbital launches will enhance generalizability and reduce false-negative classification errors.

Overall, this data science pipeline successfully bridges raw launch telemetry with actionable intelligence, demonstrating the immense value of predictive analytics in modern aerospace logistics.